# PILHA E FILA
## ENCADEADAS

Resolução de Problemas
Estruturados em Computação

Antes de entendermos como implementar as estruturas de **Pilha** e **Fila**, precisamos entender o que é o conceito de encadeamento em listas lineares.

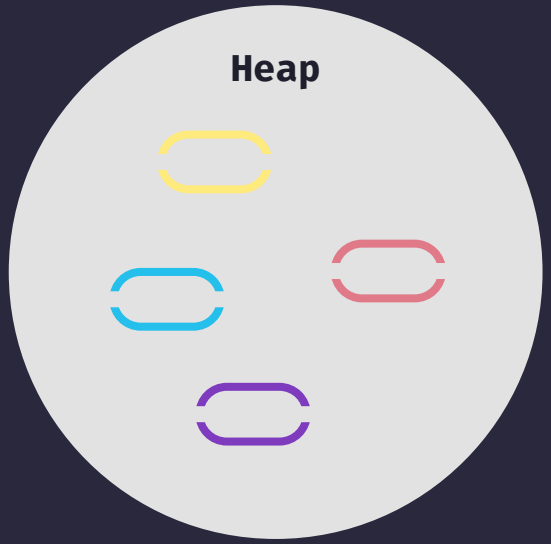
Quando estudamos sobre memória, entendemos que nossos objetos são instanciados no espaço de memória chamado **memória heap**.

Nós vimos que esse espaço de memória não possui organização no armazenamento dos dados, ou seja, os dados ficam todos “espalhados” nesse espaço de memória.

Em **estruturas encadeadas** nós instanciamos cada um dos elementos **individualmente.** Esses elementos são armazenados na **memória heap**, e para conectar esses elementos, utilizamos a técnica de encadeamento.

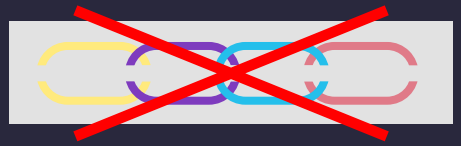
# /Encadeamento

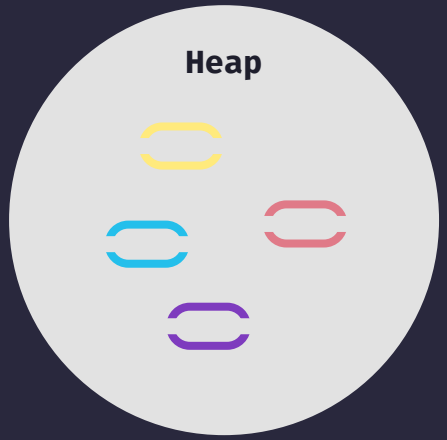
Imagine que cada elemento de uma lista linear é o elo de uma corrente:




Quando os elos de uma corrente não estão conectados, eles ficam soltos e espalhados. A mesma coisa acontece com uma lista linear. Se não conectarmos um elemento ao outro, não sabemos qual é o primeiro elemento, e nem a sequência em que os elementos foram adicionados.

# /Encadeamento






Os elementos de uma lista, nunca vão ficar lado a lado como em um array estático (lembrem que estamos falando de memória), mas conseguimos simular esse comportamento fazendo com que um elemento aponte na direção do próximo. Vamos ver como isso é organizado em termos de algoritmo

# /Encadeamento

Cada elo das nossas listas, ou “caixinha na memória”, é representado por uma estrutura chamada **nó** (node). Essa estrutura contém duas informações: o dado que está sendo inserido, e uma referência para o elemento próximo ou anterior (ou ambos).

| Node |
| --- |
| E element;<br>Node next/previous; |
| Node(E e); |

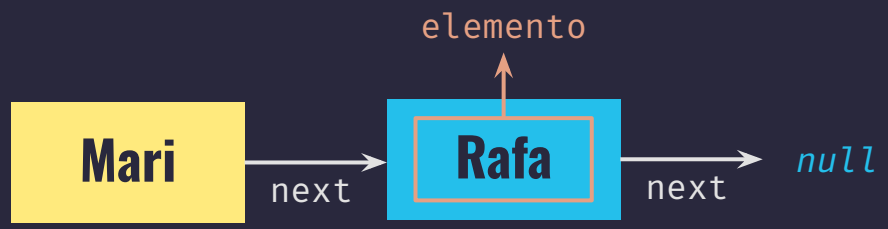
# /Adicionando Elementos

Vamos usar como exemplo a sequência de strings: “Mari”, “Rafa” e “Josi”:

A primeira string que adicionamos é “Mari”. Portanto, na nossa estrutura **Node**, o elemento será “Mari” e o next será nulo. O next é nulo pois não adicionamos nenhuma outra string à lista, portando esse **Node** não possui nenhum elemento posterior à ele.

# /Adicionando Elementos

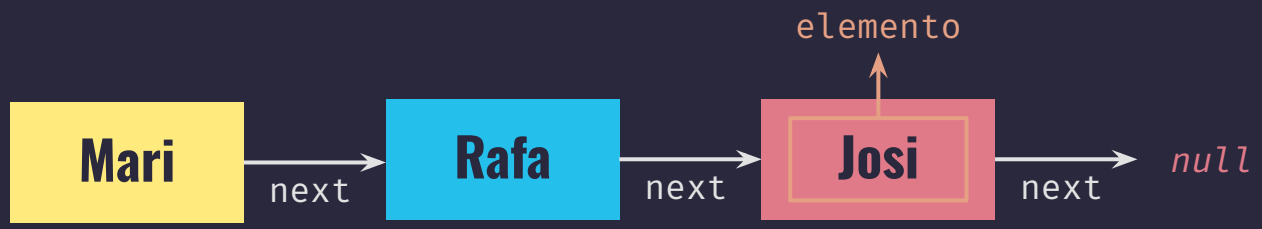
Agora vamos adicionar a string “Rafa”:




Na segunda string que adicionamos, o elemento é “Rafa” e o next é nulo, pois, da mesma que forma que no primeiro **Node**, ele não possui um elemento posterior. Mas, o **Node** do elemento “Mari” agora tem seu next apontando para o **Node** do elemento “Rafa”, pois a string “Rafa” foi adicionada posteriormente à string “Mari”.

# /Adicionando Elementos

Para fixar bem o conceito, vamos adicionar a última string (“Josi”):




A mesma situação acontece nesse exemplo e segue acontecendo enquanto strings forem adicionadas à lista. Nesta representação os elementos parecem estar lado a lado, mas na verdade eles estão espalhados na **memória heap** conforme vimos anteriormente.

# /Referência ao anterior

A ordem do encadeamento também pode ser feita de forma reversa:

A lógica do encadeamento “reverso” é a mesma, porém a referência que o **Node** vai armazenar, é uma referência para o objeto anterior.
Nós também conseguimos fazer o encadeamento duplo, mas vamos estudar isso mais pra frente.

# Pilha

# /Pilha - Lista Encadeada

A pilha é um tipo de lista encadeada que tem como princípio a sigla LIFO - Last In First Out (Último que entra é o primeiro que sai). Você pode imaginar essa estrutura como uma pilha de roupas, pratos, livros, etc. A lógica é a mesma: os elementos são empilhados na ordem em que são adicionados e, ao remover, o elemento do topo da pilha é o que será removido.

# /Pilha - Lista Encadeada

Os principais métodos de uma pilha são:

**push(E e)** Método que empilha um elemento qualquer em uma estrutura do tipo pilha. Segue a lógica de adição em uma lista encadeada que vimos nos slides anteriores.

**pop()** Método que desempilha um elemento qualquer de uma estrutura do tipo pilha. Esse método sempre vai remover e retornar o elemento no topo da pilha (último).

# /Pilha - Removendo Elementos

Para remover o **último elemento** de uma pilha encadeada:

Na pilha, sempre teremos uma referência para o objeto no topo. Essa referência sempre vai apontar para o último **Node** da pilha.
Como o método de remoção retorna o elemento do **Node**, precisamos guardar esse valor string em uma variável.

# /Pilha - Removendo Elementos

Para remover o **último elemento** de uma pilha encadeada:

Em seguida, precisamos mover o top para o **Node** anterior, e podemos definir o **Node** do elemento "Josi" como *null* (o que não é necessário pois o top já indica o final da pilha, mas não tem problema fazer também). Por fim, retornamos o elemento do **Node** removido.

# Fila

# /Fila - Lista Encadeada

A fila é um tipo de lista encadeada que tem como princípio a sigla FIFO - First In First Out (Primeiro que entra é o primeiro que sai). Você pode imaginar essa estrutura como uma fila de pessoas, pedidos, processos, etc. A lógica é a mesma: os elementos são enfileirados na ordem em que são adicionados e, ao remover, o elemento do início da fila é o que será removido.

# /Fila - Lista Encadeada

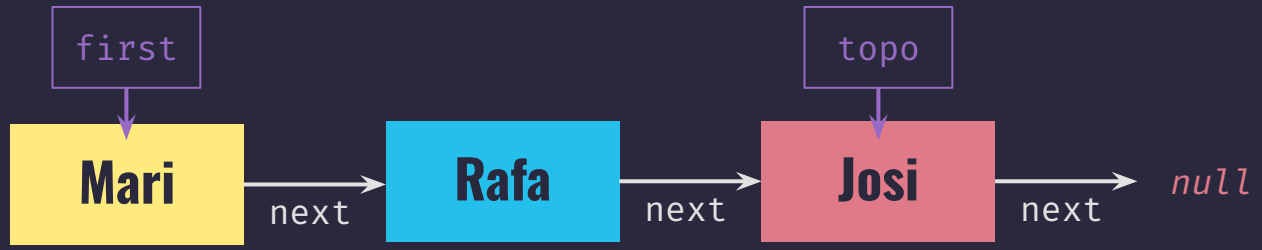
Os principais métodos de uma fila são:

**enqueue(E e)** Método que enfileira um elemento qualquer em uma estrutura do tipo fila. Segue a lógica de adição em uma lista encadeada que vimos nos slides anteriores.

**dequeue()** Método que desenfileira um elemento qualquer de uma estrutura do tipo fila. Esse método sempre vai remover e retornar o primeiro elemento da fila.

# /Fila - Removendo Elementos

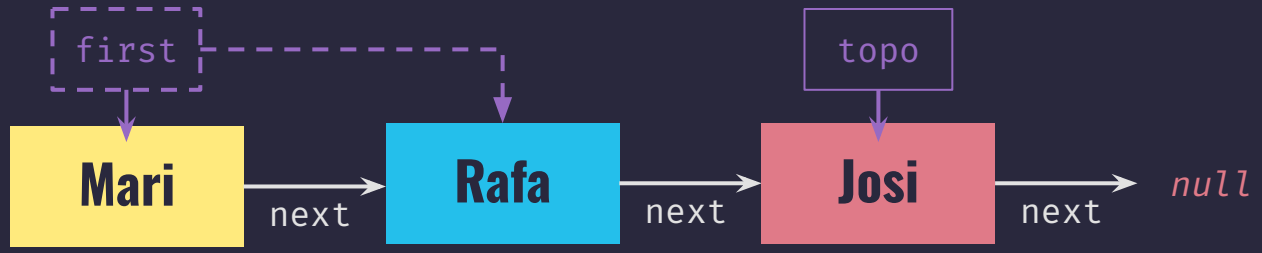
Para remover o **último elemento** de uma fila encadeada:




Na fila, além da referência no topo, temos uma referência no início. Essa referência sempre vai apontar para o primeiro **Node** da fila.
Como o método de remoção retorna o elemento do **Node**, precisamos guardar o valor string do primeiro **Node** em uma variável.

# /Fila - Removendo Elementos

Para remover o **último elemento** de uma fila encadeada:




O processo de remoção da fila é semelhante à remoção da pilha, porém, a remoção é feita pelo primeiro elemento. Então, movemos o first para o próximo **Node.**

# /ESQUELETO DA CLASSE

## Stack

```
Node top;
int size;
```

```
Stack();
push(T e);
pop();
top();
isEmpty();
size();
clear();
toArray();
toString();
```

## Queue

```
Node first
Node top;
int size;
```

```
Queue();
enqueue(T e);
dequeue();
front();
isEmpty();
size();
clear();
toArray();
toString();
```